



UNIVERSITY OF
LIVERPOOL

DEPARTMENT OF ELECTRICAL ENGINEERING & ELECTRONICS

Project Report for project 'DeepSign: Training a Convolutional Neural Network for Sign Language Understanding'

Author: Abdulaziz Almhoud (201744420)

Project Supervisor: Alessandro Varsi

Project Assessor: Lee Delvin

Declaration of academic integrity

By submitting this work, I confirm that I have read and understood the University's Academic Integrity Policy.

By submitting this work, I confirm that I have acted honestly, ethically and professionally in conduct leading to assessment for the programme of study.

By submitting this work, I confirm that the work I am submitting is my own. I have not commissioned production of the work from a third party or used artificial intelligence software in an unacceptable manner to generate the work*. I have not copied material from another person or source, nor committed plagiarism, nor fabricated, falsified or embellished data when completing the attached piece of work. I have not colluded with any other student in the preparation and/ or production of this work.

*Software applications include but are not limited to, ChatGPT, Bing Chat, DALL.E, Bard

Summary

The aim of this project is to design and implement a demo of the real-time British Sign Language Interpreter, that can be used by the hearing impaired community, which implements a robust Python to LeapC interface which shows in real-time frames, hands and fingers and a Convolutional Neural Network that can recognise at least 10 distinct British sign language hand signs. It will also include a user-friendly Python Graphical user interface,

improving the usability accessibility of the system. The Sign Language Interpreter System would be most beneficial to the hearing impaired who are reliant on communication assisting tools in their everyday life and make up, as of February 2025, over 5% of the world's population (approximately 430 million people) and it is also estimated that by 2050 every 1 in 10 people will have disabling hearing loss [1]. The system will help maintain a strong connection between people whether they are deaf or not, the world will be connected by maintaining communication.

Table of Contents

Summary	1
Project scope	3
Methodology	4
Leap Motion Controller	4
System Architecture	5
Data Acquisition	5
Preprocessing	6
Palm-relative Normalisation	6
Z-score Normalisation	6
Classification	7
Graphical User Interface	8
Results	9
Model Performance	9
Real-time Results	10
Conclusion	11
Future Work	12
Reflection	12
References	13
Appendices	14
Appendix 1. Graphical User Interface (GUI.py)	14
Appendix 2. LeapMotionApp.py	15
Appendix 3. Separate.py	18
Appendix 4. Normalisation Code (Normalisation.py)	19
Appendix 5. CNN_model.py	21
Appendix 6. Demo.py	24

Project scope

There are approximately 151,000 British Sign Language (BSL) users in the UK as estimated by the Royal National Institute for Deaf People (RNID), with 85,000 of them being deaf [2]. So, the chances of a deaf BSL user meeting another BSL user is a 0.2% chance. This shows that there is a prevalent communication gap between the hearing-impaired community and the rest of society, meaning that the hearing-impaired face communication barriers in their daily lives and as a result be socially excluded which can negatively impact their chances of employment, getting proper education and receiving proper healthcare. This communication

gap will only widen as it is estimated that by 2035, 14.2 million adults will suffer from hearing loss, further widening the gap of communication [2].

The project aims to bridge this gap of communication by developing a BSL sign language interpreter system that uses a Leap Motion Controller (LMC) to capture the user's hand signs and a Convolutional Neural Network (CNN) to interpret/classify the hand signs. The whole system implements a robust Python to LeapC interface which can recognise at least 10 distinct BSL hand signs. With the development of this system, the hearing-impaired community will be able to communicate to a wider range of people while still using their primary means of communication.

There is not an official product that makes use of machine learning to allow deaf people and hearing people to be able to communicate with each other in real-time. Instead, there are systems that can translate videos or announcements provided by companies such as Signapse, however, such systems cannot be used by a deaf BSL user to help them communicate with another hearing person in real-time. Another way is that a deaf person may hire or book an interpreter, Lingo Service provide a way to book a person to help interpret for the deaf person. However, costs can be quite high to hire for short periods of time (£175 for 3 hours). There have been small projects that have attempted to solve the problem of the communication gap between the hearing impaired and the rest of society, but no such system has been commercialised.

Methodology

Leap Motion Controller

The dataset was collected using a Leap Motion Controller (LMC) developed by Leap Motion. The choice of the LMC as the sensor was due to its tracking capabilities, being able to provide highly accurate and structured hand and finger skeletal data which is stored in the dataset and fed to the CNN. It uses two monochromatic infrared cameras and three infrared LEDs as shown in Figure 2.

The LMC comes with an API that allows you to get metadata and extract hand features. With the use of a GitHub repository from Leap Motion, which allows the use of Python 3 and provides a beginning code, `tracking_event.py` (Appendix 3), that serves as a beginning to extract features from the LMC such as the palm position, number of hands and hand type [4]. The code was then extended to include more features which are palm normal, joint coordinates, fingertip coordinates and extend states (Appendix 4).

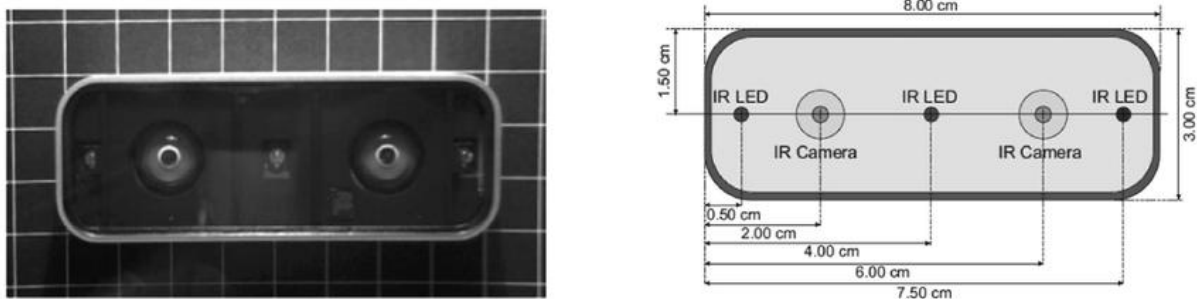


Figure 1: Leap Motion Controller and Schematic

System Architecture

System was developed in 4 main stages. In the first stage, data acquisition, the Leap Motion Controller is used to detect the hand sign the user presents, then using the Leap API, the features of the hand are extracted. In the preprocessing stage, the extracted features are then pre-processed for CNN learning. The pre-processed features are then passed onto the CNN as inputs which the CNN learns patterns from and is then able to classify the hand sign shown.

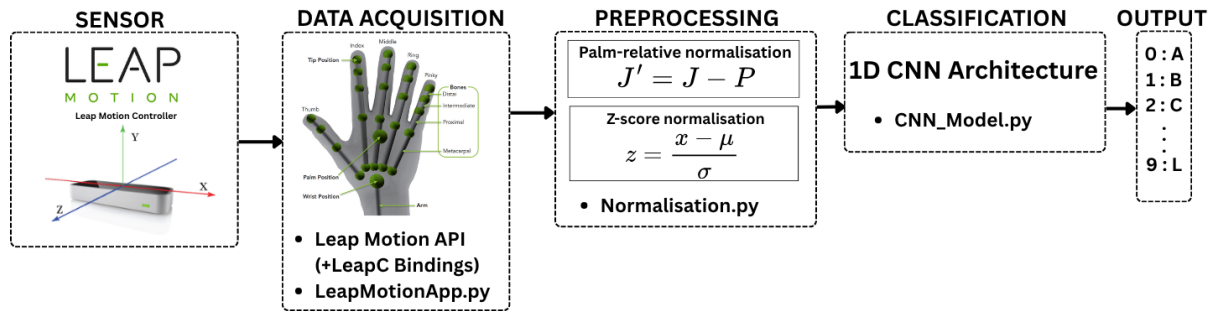


Figure 2: Proposed System

Data Acquisition

The LMC was used to capture 10 distinct BSL hand signs, letters A to L excluding H and J as they are dynamic, meaning they require movement during capture. The dataset contained a total of 733 samples with each hand sign having around 60-90 samples depending on how difficult the hand sign is to capture and any similarities with other hand signs in the dataset. Each sample contained 297 features:

1. **Palm position (x, y, z):** the 3D coordinates of the palm centre in the plane (i.e., where the palm is above the LMC).
2. **Palm normal (x, y, z):** a unit vector perpendicular to the palm surface that indicates the orientation of the hand (tells you which way the palm is facing).
3. **Number of hands:** indicates the number of hands detected.
4. **Hand type:** identifies which hand is detected (left or right)
5. **Finger joint coordinates (x, y, z):** the 3D coordinates of the finger joints for each joint, which include the metacarpal, proximal, intermediate and distal. This provides information on finger curvature, bending angles and relative position between joints.
6. **Fingertip coordinates (x, y, z):** the 3D coordinates of the fingertips, which are basically the end of the distal joint.
7. **Finger extended states:** indicates whether a finger is extended or not. Helps CNN quickly distinguish between hand signs that have many extended fingers such as the hand sign for A or hand signs that have no extended fingers such as B.

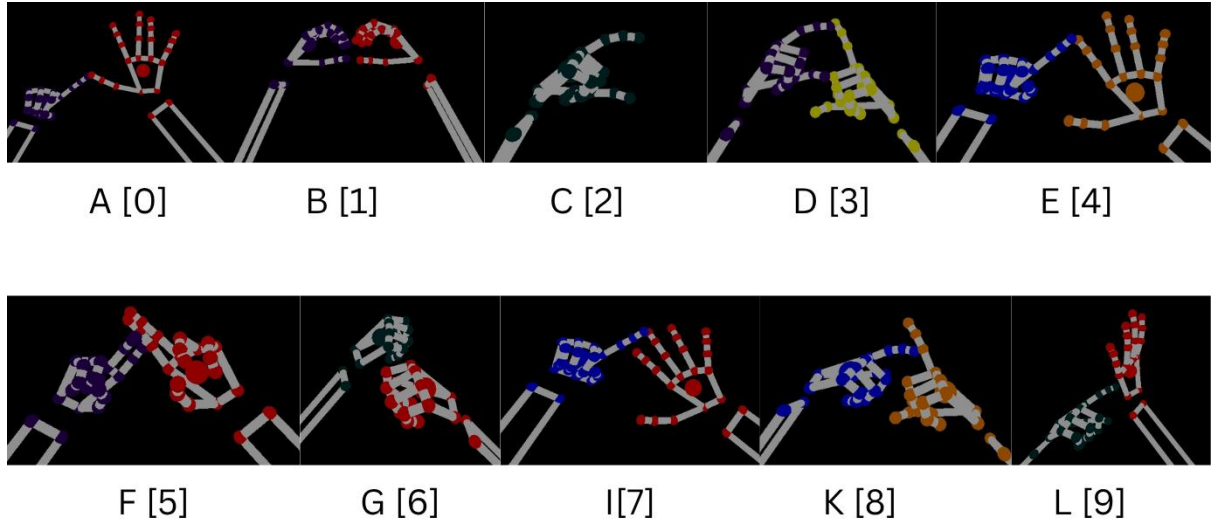


Figure 3: Mirrored Hand signs performed above leap motion controller labelled with letter and class

Preprocessing

Since the model trained via supervised learning, the dataset the labels (hand signs) and the features were separated so the CNN model can learn to map the input features to the labels. Then any features that involved text in the dataset were converted into numbers, features such as the hand type were changed to 0 (left) and 1 (right) and extended states changed to 0 (not extended) and 1 (extended). Afterwards, two preprocessing techniques were applied using pandas to the dataset to help speed up and improve learning for more accurate classification of the hand signs by ensuring that learning is not by unimportant patterns or dominated by certain features (Appendix 4).

Palm-relative Normalisation

If the same gesture, for example A, was performed multiple times at different positions, each sample would have very different values for the palm position (-650 to 600), the CNN model would treat these variations as meaningful differences and will therefore cause the CNN to misclassify the hand signs, classifying the same hand sign as different one due to its position. Palm-relative normalisation was applied to all the finger joint positions, setting the finger joint coordinates relative to the palm, ensuring that the CNN model learns the shape of the hand detected instead of having learning driven by where the hand is above the sensor. This was done by taking away the palm position away from the joint coordinates and shifting the origin from the sensor's reference frame to the centre of the palm.

$$J' = J - P$$

$$P = 0$$

Z-score Normalisation

Some features, such as palm normal and finger joint positions, have a larger scale that can range from -650 to +650 as compared to features such as the number of hands, hand type and finger extend state which only range from 0 to 2 as values. that could dominate learning, meaning that the CNN could give more importance to the feature with the larger scale. For example, the signs for C and D are quite similar except that D requires both hands while C

only needs one. If the sign for D was displayed, it may be misclassified as C as one of the hands in D is the sign for C, as the number of hands feature won't be taken into account as much as the joint positions. So, the features with large scales, such as palm normal, finger joint positions and fingertip positions, are standardised transforming the data so that they have a mean of zero and a standard deviation of one. Now, all the features are on the same scale, with the range of values now being around -5 to +5. By scaling features to a common range, no feature dominates the learning, which enhances the accuracy of the CNN model leading to less misclassifications.

$$z = \frac{x - \mu}{\sigma}$$

Classification

After preprocessing, the features are passed onto a one-dimensional Convolutional Neural Network (CNN) as a 297-dimensional feature vector representing hand geometry which is reshaped into a one-dimensional feature sequence ($x \in \mathbb{R}^{297 \times 1}$). A 1D CNN, as shown in figure 5, was used because data used consists of a structured one-dimensional feature vector describing hand geometry. Therefore, a 1D CNN is the most appropriate, as it can directly process sequential numerical features without requiring reshaping into image-like formats. 1D CNN efficiently captures relationships between neighbouring features. The 1D CNN architecture allows effective classification by transforming the raw feature data into higher-level representations.

In the first convolutional layer, 64 filters slide across the input feature vector where each filter computes a weighted combination of neighbouring features. It allows the network to learn low-level patterns of 3 neighbouring features, patterns such as finger extension combinations and local finger curvature using the finger joint coordinates or correlations between finger extension states and joint coordinates, they are basic patterns that allow the CNN to identify finger curvature. The learned patterns are passed through ReLU activation, introducing non-linearity, which allows the CNN to learn complex, non-linear patterns. The relationship between joint coordinates and hand gestures is non-linear, as a small change in joint position or hand orientation can determine what hand sign is classified. ReLU helps the CNN distinguish between hand signs that have subtle differences such as A, E and I, that only differ by finger joint, fingertip coordinates and hand orientation.

$$ReLU \rightarrow f(x) = \max(0, x)$$

The second convolutional layer is nearly identical to the first except there are twice as many filters which are used to learn more complex, higher-level combinations between neighbouring patterns. While the first layer captures simple geometric relationships, the second layer combines these into higher-level features such as overall hand shape, finger coordination patterns, and spatial relationships between multiple fingers.

After the convolutional layers, the feature maps are flattened, converting them into a one-dimensional vector that summarises the hand shape geometry. This extracted feature vector

contains the most important patterns learned in the convolutional layers which include finger bending patterns, hand orientation capture by palm normal and the overall hand structure

The flattened patterns are passed into a fully connected dense layer with 64 neurons. Each neuron computes a weighted sum combination followed by ReLU. As shown in figure 5 in the fully connected layer, each neuron is connected to all inputs from the previous layer, allowing the model to combine the extracted features to form high-level representations. This stage effectively learns which combinations of features correspond to specific gestures, it allows the CNN to distinguish between gestures base on their overall handhape rather than isolated features. For example, a particular combination of finger bending and joint positions may strongly indicate a specific hand sign or curved fingers forming a circular shape will be classified as C. Also, by randomly disabling 30% of neurons during training overfitting will be reduced, making sure the model learns more generalised patterns rather than memorising the training data.

Finally, the output layer consists of a dense layer with ten neurons equal to the number of hand sign classes, using a softmax activation function. This produces a probability distribution over all possible classes, where each value represents the likelihood of the input gesture matching to a particular hand sign. The predicted hand sign is determined by selecting the class with the highest probability.

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

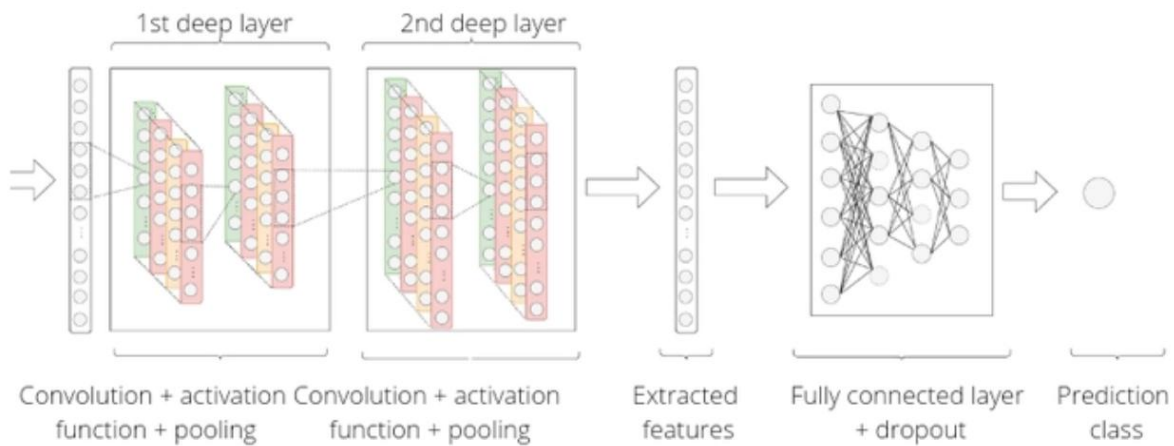


Figure 4: 1D CNN Architecture [7]

Graphical User Interface

The initial objective was to have a robust, user-friendly Python graphical user interface GUI to show the output of the system, which is the conversion of the hand signs to letters and in the future words and sentences, in a visually pleasing format. The prototype for the GUI was developed, however it was not implemented into the system due to a lack of time in understanding how to include it at the end of the project.

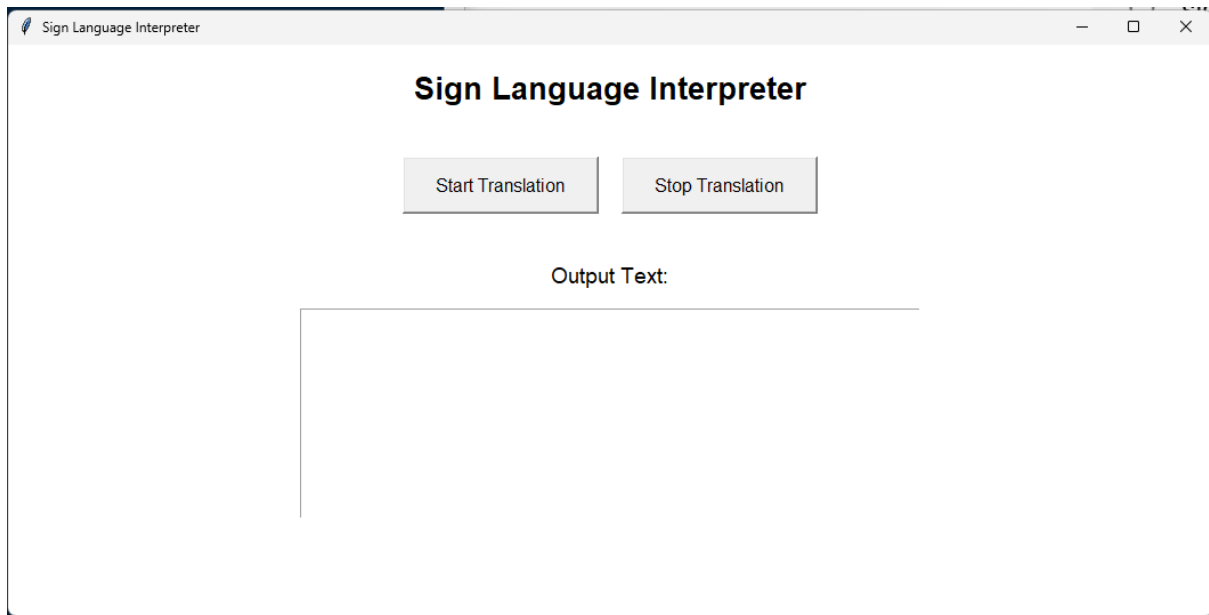


Figure 5: Python Graphical User Interface

Results

Model Performance

80% of the dataset was used to train the CNN model and 20% of that was used to validate it. As shown in figure 6, performance of the CNN rapidly converged within the first two epochs, with training accuracy rising from 60% to 88%, which shows that the CNN was able to quickly learn the patterns of the features provided in the dataset. Validation accuracy reached 99%, indicating that the CNN did not just memorise specific features and instead was able to effectively generalise to unseen data.

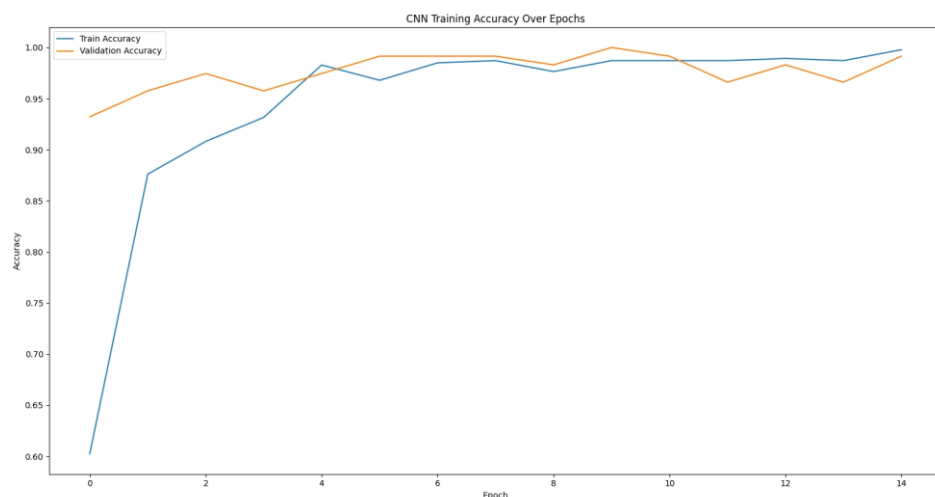


Figure 6: CNN Performance

20% of the dataset was used for testing the model and as can be seen in figure 7, the confusion matrix shows that the CNN achieved 100% test accuracy. The CNN correctly classified all hand signs.

These near perfect results are likely due to two main reasons:

1. By applying the **preprocessing techniques** to the dataset, it ensured that the CNN would focus on learning important features and patterns such as the overall handshape and not where the hand is positioned above the sensor. It also avoided the dominance of certain feature with larger scales, enhancing the CNN's accuracy.
2. **The dataset captured was too clean and small.** There was only one subject performing the hand signs and each sample was captured with consistent hand positioning with little to no differences between them meaning most samples for a hand sign were near identical. Also, since there were only ten classes, the classification task was quite simple as the CNN would only need to identify from a small number of hand signs.

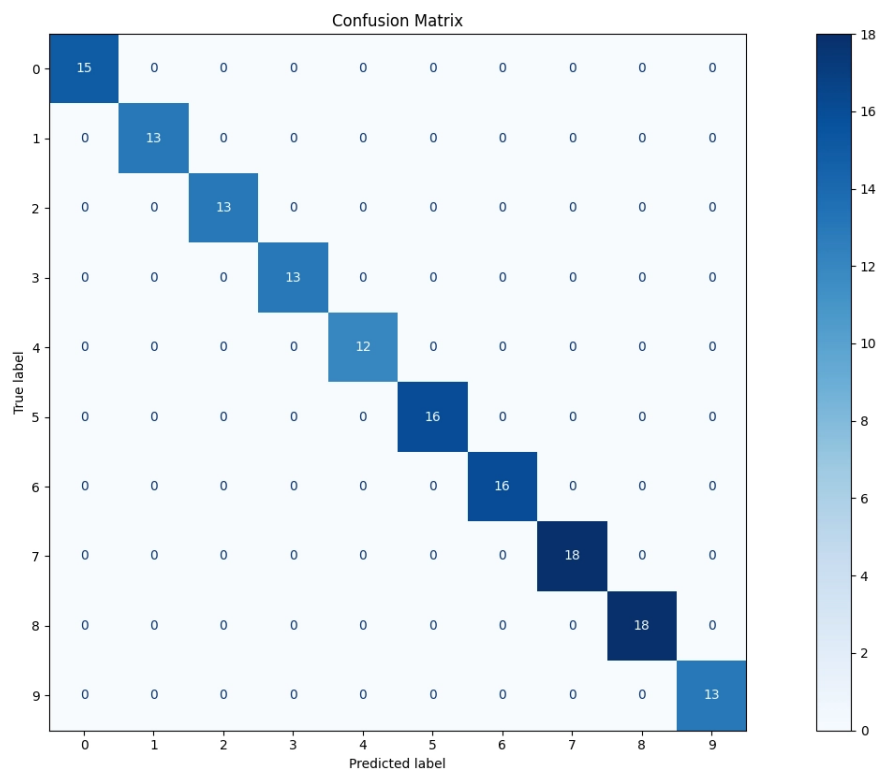


Figure 7: Confusion Matrix - shows the classification accuracy of the CNN

Real-time Results

To evaluate the practical performance of the system, real-time experiments were conducted in which 10 tests were performed for each hand sign. The results, shown in table 1, demonstrate that the system performs well in real-world conditions, achieving an overall accuracy of approximately 88% with a response time of around two seconds.

As can be seen from table 1, hand signs A, B, C, D and F were classified with 100% accuracy. C was not misclassified most likely due to it being the only hand sign that only needs a single hand. The rest of the hand signs, excluding A and D, have distinct features as compared to the rest which was why they were not confused with other hand signs.

A, E, and I were occasionally confused due to similarities in finger bending patterns and overall hand structure. In many cases, the model showed a bias towards predicting class A, suggesting that its features were more dominant or frequently represented in the dataset. This was likely due to how the hand, with all fingers extended, was oriented. The more tilted the hand was (fingers pointing towards the other hand) the more likely it was to be E or I, but during testing I did not tilt my hand so, the CNN misclassified them as A. So, the misclassifications here were not caused mainly by similarities but by the CNN focusing too much on the orientation of the hand

Hand sign K was getting confused with B, which were similar in terms of the finger coordinates and finger extend states as well as identical in hand orientation. L was getting misclassified as C due to the LMC not being able to accurately capture the one of the hands leading to only one hand, that looks like C, to be detected.

Table 1: Real-time accuracy test results

Hand sign	Accuracy
A	100%
B	100%
C	100%
D	100%
E	70%
F	100%
G	80%
I	70%
K	80%
L	80%

Overall, the system achieved high accuracy with relatively fast response times. However, these results highlight limitations in both the LMC's detection accuracy and the lacking dataset, that contained a small number of samples for each hand sign that were indistinguishable. This system has not been tested by anyone other than I, who captured the samples, meaning the system may not perform as well when tested by others, as there will be differences in joint coordinates and how the hand is oriented when performing the hand signs.

Conclusion

Sign Language is essential tool for the hearing-impaired to be able to communicate with the rest of society and to go about in their day-to-day lives. However, not many people, that are not deaf, know sign language unless they have a close relative that is, meaning hearing-impaired individuals are unlikely to be able to communicate with people they have not met before. This project presents a method for British Sign Language recognition using a Leap Motion Controller to capture skeletal data with high accuracy, then extract the feature data using its API. The extracted features are then pre-processed to ensure the CNN makes proper use of the dataset during learning. Then the dataset is fed into the 1D CNN, which learns

patterns from the features, allowing it to accurately classify hand signs in real-time with 88% accuracy in about two seconds which meets my initial objective of achieving a CNN with at least 85% accuracy. The dataset was also captured with enough classes and samples meeting the objective I originally planned, but I should have captured more samples as to improve the classification accuracy. Of the four small objectives I set out, three of them were complete excluding the Python GUI, which was not implemented into the system due to a lack of time to learn how. The major objective, which was the development of a system capable of sign language recognition, I felt as though I achieved a working, very early prototype that can be used in understanding how various features of the hand and how normalisation improves CNN classification, but not a final product that can be used.

Future Work

With more time and resources available to me, I would have improved the system by strengthening the CNN's classification accuracy, having up to date hardware and improving the user-friendliness of the system. I will do this by:

- **Capturing a more robust dataset:** by taking capturing samples from multiple subjects, as well as having an increased number of samples. This can significantly enhance the CNN's classification performance by providing the model with more diverse and nuanced representations of the data as compared to the current dataset. This diversity helps the model to perform much better in real-time applications by various people.
- **Leap Motion Controller 2:** by investing in a more up to date sensor, the system's accuracy will significantly improve, freeing it from the limitation of detection. As I previously mentioned a lot of the time hands would not be detected even when in clear view, which caused the CNN to misclassify the gestures, even when they were performed correctly and the CNN has already learned what hand sign is being performed.
- **Having the system be more user-friendly:** by improving on the unfinished GUI (better visuals) and including it, the user will be able to present the screen to others when trying to communicate with non-BSL users. I would also make it so the system can output using speech, making it seem as if the user is talking. I would also make the system more portable, as the user is more likely to communicate with people who don't know sign language outside the comfort of their own home, around family. I would do this by using a raspberry pi which would be connected to the LMC instead of a laptop.

Reflection

This project has given me a much better understanding of how machine learning process works, more specifically CNN learning. I realised how heavily reliant a CNN is on the data or dataset provided; a dataset with a small number of samples and a small number of classes is just as likely to misclassify as a dataset with a larger number of samples and classes during training. But this does not mean that the smaller dataset will achieve a similar or higher accuracy during real-time applications, which can be affected by various factors such as the subject that is being captured can have different sized hands leading to different joint

coordinates and there possibly lead to misclassifications, also the hand signs can be performed slightly different for each person in terms of how the hand is oriented, which makes the classification problem more difficult.

The most difficult part of the project was when I was setting up the LMC and it's API and had it work with Python. Due to the lack of documentation on my end, I forgot how I initially set up the Python to Leap Motion interface that allowed me to be able to extract feature data from the leap motion controller using Python 2. I left my work for a while and when I came back, the code was not working, and I still do not know why. So, I had to repeat it again but this time using a Python wrapper from GitHub repository by Ultraleap I was able to have it have the interface work with Python 3, which made it more difficult than before as there was no official documentation for it.

Overall, this project helped me have a better understanding of machine learning, allowing to feel more prepared to study Data Science and AI masters next year.

References

- [1] World Health Organization, "Deafness and hearing loss," Fact sheet, 26 Feb. 2025. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/deafness-and-hearing-loss>. [Accessed: 1 Oct. 2025].
- [2] GOV.UK, "The British Sign Language (BSL) report 2022" Available: <https://www.gov.uk/government/publications/the-british-sign-language-bsl-report-2022/the-british-sign-language-bsl-report-2022> [Accessed: 1 Oct. 2025].
- [3] Ultraleap, "LeapC Guide – Ultraleap Tracking API." [Online]. Available: <https://docs.ultraleap.com/api-reference/tracking-api/leapc-guide.html>. [Accessed: 1 Oct. 2025].
- [4] GitHub, "leapc-python-bindings" [Online]. Available: <https://github.com/ultraleap/leapc-python-bindings> [Accessed: 1 Jan. 2026].
- [5] Ultraleap, "API Overview — Leap Motion Python SDK v3.2 Beta documentation." [Online]. Available: https://www.leapmotiontechnology.com/developer-archive-sub/documentation/python/devguide/Leap_Overview. [Accessed: 1 Oct. 2025].
- [6] Big Vision, "Implementing a CNN in TensorFlow & Keras" [Online]. Available: <https://learnopencv.com/implementing-cnn-tensorflow-keras/>. [Accessed: 12 Feb. 2026].
- [7] Keras, "Developer Guides", [Online]. Available: https://keras.io/guides/functional_api/. [Accessed 12 Feb. 2026]

Appendices

Appendix 1. Graphical User Interface (GUI.py)

```
import tkinter as tk
```

```
class SignLanguageGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Sign Language Interpreter")
        self.root.geometry("700x500")
        self.root.configure(bg="white")

        self.title_label = tk.Label(
            self.root,
            text="Sign Language Interpreter",
            font=("Arial", 20, "bold"),
            bg="white",
            fg="black"
        )
        self.title_label.pack(pady=20)

        self.button_frame = tk.Frame(self.root, bg="white")
        self.button_frame.pack(pady=20)

        self.start_button = tk.Button(
            self.button_frame,
            text="Start Translation",
            font=("Arial", 12),
            width=18,
            height=2,
            command=self.start_translation
        )
        self.start_button.grid(row=0, column=0, padx=10)

        self.stop_button = tk.Button(
            self.button_frame,
            text="Stop Translation",
            font=("Arial", 12),
            width=18,
            height=2,
            command=self.stop_translation
        )
        self.stop_button.grid(row=0, column=1, padx=10)

        self.output_label = tk.Label(
```

```

        self.root,
        text="Output Text:",
        font=("Arial", 14),
        bg="white",
        fg="black"
    )
    self.output_label.pack(pady=(20, 5))

    self.output_box = tk.Text(
        self.root,
        height=10,
        width=60,
        font=("Arial", 12),
        wrap="word"
    )
    self.output_box.pack(pady=10)

    def start_translation(self):
        self.output_box.insert(tk.END, "Translation started...\n")

    def stop_translation(self):
        self.output_box.insert(tk.END, "Translation stopped.\n")

root = tk.Tk()
app = SignLanguageGUI(root)
root.mainloop()

```

Appendix 2. LeapMotionApp.py

```

import leap
import time
import keyboard

class MyListener(leap.Listener):
    def __init__(self):
        print("Initialized")

        super().__init__()
        self.last_print_time = 0.0

    self.capture_key = "s"
    self.dataset_file = open(r"C:\Development\leapc-python-bindings-
main\Dataset\Handsigns.csv", "a")

```

```

self.current_label = "L" # Preprocess

def on_connection_event(self, event):
    print("Leap Service Connected")

def on_connection_lost_event(self, event):
    print("Leap Service Disconnected")

def on_device_event(self, event):
    try:
        with event.device.open():
            info = event.device.get_info()
    except leap.LeapCannotOpenDeviceError:
        info = event.device.get_info()

    print("Connected")
    print(f"Found device {info.serial}")

def on_device_lost_event(self, event):
    print("Disconnected")

def on_exit_event(self, event):
    print("Exited")

def on_tracking_event(self, event):
    current_time = time.time()

    # Print every 0.2 seconds
    if current_time - self.last_print_time < 0.2:
        return

    self.last_print_time = current_time

    # Capture only if at least 1 hand and s pressed
    if not keyboard.is_pressed(self.capture_key):
        return

    if len(event.hands) < 1:
        return

```

```

# Timestamp
ts_us = event.info.timestamp

# Sample
output = []

output.append(self.current_label)

# Frame info
output.append(str(event.tracking_frame_id))
output.append(str(ts_us))
output.append(str(ts_s))
output.append(str(len(event.hands)))

# Hand info (type, confidence, palm, fingers and bones)
for hand in event.hands:

    hand_type = "left" if str(hand.type) == "HandType.Left" else "right" # Preprocess
    hand_confidence = hand.confidence
    normal = hand.palm.normal
    palm = hand.palm.position

    output.append(hand_type)
    output.append(str(hand_confidence))

    output.extend([
        str(palm.x), str(palm.y), str(palm.z),
        str(normal.x), str(normal.y), str(normal.z)
    ])

# Fingers and Bones
finger_names = ['Thumb', 'Index', 'Middle', 'Ring', 'Pinky']
bone_names = ['Metacarpal', 'Proximal', 'Intermediate', 'Distal']

for i, digit in enumerate(hand.digits):
    name = finger_names[i] if i < len(finger_names) else f'digit_{i}'

    # Fingertip positions
    tip = digit.distal.next_joint
    output.extend([str(tip.x), str(tip.y), str(tip.z)])
    bones = [digit.metacarpal, digit.proximal, digit.intermediate, digit.distal]

    for bone_name, bone in zip(bone_names, bones):

        pj = bone.prev_joint
        nj = bone.next_joint

```

```

        output.extend([
            str(pj.x), str(pj.y), str(pj.z),
            str(nj.x), str(nj.y), str(nj.z)
        ])

    # Extended state
    is_extended = digit.is_extended
    output.append("EXTENDED" if is_extended else "NOT EXTENDED") #
Preprocess

    row = ",".join(output)
    print(row)
    self.dataset_file.write(row + "\n")

def main():
    my_listener = MyListener()

    connection = leap.Connection()

    connection.add_listener(my_listener)

    running = True

    with connection.open():
        connection.set_tracking_mode(leap.TrackingMode.Desktop)
        while running:
            time.sleep(1)

if __name__ == "__main__":
    main()

```

Appendix 3. Separate.py

```

import pandas as pd

file_path = r"C:\Development\leapc-python-bindings-main\Dataset\Handsigns.csv"

df = pd.read_csv(file_path, header=None)

# Separate labels and features
y = df.iloc[:, 0]
X = df.iloc[:, 1:]

```

Appendix 4. Normalisation Code (Normalisation.py)

```
import pandas as pd

input_file = r"C:\Development\leapc-python-bindings-main\Dataset\Handsigns.csv"
output_file = r"C:\Development\leapc-python-bindings-main\Dataset\Handsigns_preprocessed1.csv"

means_file = r"C:\Development\leapc-python-bindings-main\Dataset\zscore_means.csv"
stds_file = r"C:\Development\leapc-python-bindings-main\Dataset\zscore_stds.csv"

df = pd.read_csv(input_file, header=None)

# Metadata
label = 0
frame = 1
time_us = 2
time_s = 3
num_hands = 4

each_hand = 148 # No. columns for each hand
hand1_start = 5
hand2_start = hand1_start + each_hand # 153

# Hand block end columns
hand1_end = hand1_start + each_hand - 1 # 152
hand2_end = hand2_start + each_hand - 1 # 300

# Palm columns
palm1_x = hand1_start + 2 # 7
palm1_y = hand1_start + 3 # 8
palm1_z = hand1_start + 4 # 9

palm2_x = hand2_start + 2 # 155
palm2_y = hand2_start + 3 # 156
palm2_z = hand2_start + 4 # 157

df.loc[df[num_hands] == 1, hand2_start:hand2_end] = 0

def feature_cols(hand_start):
    cols = []

    finger_start = hand_start + 8

    each_finger = 28
```

```

# Palm normal columns
normal_cols = [hand_start + 5, hand_start + 6, hand_start + 7]

for finger_id in range(5):
    start = finger_start + finger_id * each_finger # Thumb = 13   Index = 41   Middle = 69
    Ring = 97   Pinky = 125

    # Tip posisitons (x,y,z)
    cols.extend([start, start + 1, start + 2]) # 13, 14, 15

    # Bone joints
    cols.extend(range(start + 3, start + 27)) # 16-40

return cols, normal_cols

hand1_cols, hand1_normal_cols = feature_cols(hand1_start)
hand2_cols, hand2_normal_cols = feature_cols(hand2_start)

# Palm-relative Normalisation
# Hand 1
for i in range(0, len(hand1_cols), 3):
    x_col = hand1_cols[i]
    y_col = hand1_cols[i + 1]
    z_col = hand1_cols[i + 2]

    df[x_col] = df[x_col] - df[palm1_x]
    df[y_col] = df[y_col] - df[palm1_y]
    df[z_col] = df[z_col] - df[palm1_z]

df[palm1_x] = 0
df[palm1_y] = 0
df[palm1_z] = 0

# Hand 2
for i in range(0, len(hand2_cols), 3):
    x_col = hand2_cols[i]
    y_col = hand2_cols[i + 1]
    z_col = hand2_cols[i + 2]

    df[x_col] = df[x_col] - df[palm2_x]
    df[y_col] = df[y_col] - df[palm2_y]
    df[z_col] = df[z_col] - df[palm2_z]

if palm2_z < df.shape[1]:

```

```

df[palm2_x] = 0
df[palm2_y] = 0
df[palm2_z] = 0

# Z-score Normalisation
zscore_cols = (
    hand1_cols + hand1_normal_cols + hand2_cols + hand2_normal_cols
)

zscore_cols = [col for col in zscore_cols if col < df.shape[1]]

means = df[zscore_cols].mean()
stds = df[zscore_cols].std()

stds = stds.replace(0, 1)

means.to_csv(means_file, index=False, header=False)
stds.to_csv(stds_file, index=False, header=False)

print("Saved z-score means to:", means_file)
print("Saved z-score stds to:", stds_file)

df[zscore_cols] = (df[zscore_cols] - means) / stds

df.to_csv(output_file, index=False, header=False)

```

Appendix 5. CNN_model.py

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import tensorflow as tf # Deep learning framework
from tensorflow import keras # Keras API for model building
from tensorflow.keras import layers # Layers for the neural network
from tensorflow.keras.callbacks import EarlyStopping # Stop training
import matplotlib.pyplot as plt # For visualizing training progress

# Load dataset
input_file = r"C:\Development\leapc-python-bindings-main\Dataset\Handsigns_preprocessed.csv"

```

```

df = pd.read_csv(input_file, header=None)

print("Dataset shape:", df.shape)

# Separate labels and features
y = df.iloc[:, 0].values.astype("int32")
X = df.iloc[:, 4:].values.astype("float32")

print("Features shape:", X.shape)
print("Label shape:", y.shape)

# Train/test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)

# Prepare features (X) and labels (y) for training a 1D CNN
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)

# 1D CNN Model -> CHECK FILTER VALUES!
num_classes = len(np.unique(y))
input_shape = (X_train.shape[1], 1)

model = keras.Sequential([
    keras.Input(shape=input_shape),

    # First Layer
    layers.Conv1D(filters=64, kernel_size=3, activation='relu'),
    layers.MaxPooling1D(pool_size=2),

    # Second Layer
    layers.Conv1D(filters=128, kernel_size=3, activation='relu'),
    layers.MaxPooling1D(pool_size=2),

    # Flatten Layer
    layers.Flatten(),

```

```

# Fully Connected Dense Layer
layers.Dense(64, activation='relu'),

# Dropout Layer
layers.Dropout(0.3),

# Fully Connected (Output) Layer
layers.Dense(num_classes, activation='softmax')
])

# Compile the Model -> CHECK LOSS!
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy']
)

model.summary()

# Callbacks
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

# Train the CNN Model -> CHECK VALIDATION!
history = model.fit(X_train, y_train, epochs=30, batch_size=16, verbose=1,
                    validation_split=0.2, callbacks=[early_stopping])

# Evaluate the Model Performance
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=1)
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Loss: {test_loss:.4f}")

# Confusion Matrix
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

```

```

disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

# Training Performance Visualised
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('CNN Training Accuracy Over Epochs')
plt.show()

plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('CNN Training Loss Over Epochs')
plt.show()

# Save the Model
model.save(r"C:\Development\leapc-python-bindings-main\Dataset\BSL_CNN.keras")
print("Model saved successfully.")

```

Appendix 6. Demo.py

```

import leap
import time
import numpy as np
import pandas as pd
from tensorflow import keras

class MyListener(leap.Listener):
    def __init__(self):
        print("Demo initialized")

        super().__init__()
        self.last_print_time = 0.0

    # Load CNN model

```

```

self.model_path = r"C:\Development\leapc-python-bindings-main\CNN
model\BSL_CNN.keras"
self.model = keras.models.load_model(self.model_path)

# Load z-score means and standard deviations
self.means_file = r"C:\Development\leapc-python-bindings-
main\Dataset\zscore_means.csv"
self.stds_file = r"C:\Development\leapc-python-bindings-main\Dataset\zscore_stds.csv"

self.means = pd.read_csv(self.means_file, header=None).iloc[:,
0].to_numpy(dtype="float32")
self.stds = pd.read_csv(self.stds_file, header=None).iloc[:,
0].to_numpy(dtype="float32")

# Label mapping
self.label_map = {
    0: "A",
    1: "B",
    2: "C",
    3: "D",
    4: "E",
    5: "F",
    6: "G",
    7: "I",
    8: "K",
    9: "L"
}

# Column layout
self.label = 0
self.frame = 1
self.time_us = 2
self.time_s = 3
self.num_hands = 4

self.each_hand = 148
self.hand1_start = 5
self.hand2_start = self.hand1_start + self.each_hand

self.hand1_end = self.hand1_start + self.each_hand - 1
self.hand2_end = self.hand2_start + self.each_hand - 1

self.palm1_x = self.hand1_start + 2
self.palm1_y = self.hand1_start + 3
self.palm1_z = self.hand1_start + 4

```

```

self.palm2_x = self.hand2_start + 2
self.palm2_y = self.hand2_start + 3
self.palm2_z = self.hand2_start + 4

#Z-score columns
self.hand1_cols, self.hand1_normal_cols = self.feature_cols(self.hand1_start)
self.hand2_cols, self.hand2_normal_cols = self.feature_cols(self.hand2_start)

self.zscore_cols = (
    self.hand1_cols + self.hand1_normal_cols +
    self.hand2_cols + self.hand2_normal_cols
)

print("CNN model loaded")

def on_connection_event(self, event):
    print("Leap Service Connected")

def on_connection_lost_event(self, event):
    print("Leap Service Disconnected")

def on_device_event(self, event):
    try:
        with event.device.open():
            info = event.device.get_info()
    except leap.LeapCannotOpenDeviceError:
        info = event.device.get_info()

    print("Connected")
    print(f'Found device {info.serial}')

def on_device_lost_event(self, event):
    print("Disconnected")

def on_exit_event(self, event):
    print("Exited")

# Normalisation.py
def feature_cols(self, hand_start):
    cols = []

```

```

finger_start = hand_start + 8
each_finger = 28

# Palm normal columns
normal_cols = [hand_start + 5, hand_start + 6, hand_start + 7]

for finger_id in range(5):
    start = finger_start + finger_id * each_finger

    # Tip positions (x,y,z)
    cols.extend([start, start + 1, start + 2])

    # Bone joints
    cols.extend(range(start + 3, start + 27))

return cols, normal_cols

def on_tracking_event(self, event):
    current_time = time.time()

    if current_time - self.last_print_time < 0.5:
        return

    self.last_print_time = current_time

    # Must detect at least one hand
    if len(event.hands) < 1:
        return

    # Timestamp
    ts_us = event.info.timestamp
    ts_s = ts_us / 1_000_000.0

    # Sample
    output = []

    # Dummy label placeholder
    output.append("-1")

    # Frame info
    output.append(str(event.tracking_frame_id))
    output.append(str(ts_us))
    output.append(str(ts_s))
    output.append(str(len(event.hands)))

```

```

# Hand info (type, confidence, palm, fingers and bones)
for hand in event.hands:
    hand_type = "0" if str(hand.type) == "HandType.Left" else "1"
    hand_confidence = hand.confidence
    normal = hand.palm.normal
    palm = hand.palm.position

    output.append(hand_type)
    output.append(str(hand_confidence))

    output.extend([
        str(palm.x), str(palm.y), str(palm.z),
        str(normal.x), str(normal.y), str(normal.z)
    ])

# Fingers and Bones
finger_names = ['Thumb', 'Index', 'Middle', 'Ring', 'Pinky']
bone_names = ['Metacarpal', 'Proximal', 'Intermediate', 'Distal']

for i, digit in enumerate(hand.digits):
    name = finger_names[i] if i < len(finger_names) else f'digit_{i}'

    # Fingertip positions
    tip = digit.distal.next_joint
    output.extend([str(tip.x), str(tip.y), str(tip.z)])

    bones = [digit.metacarpal, digit.proximal, digit.intermediate, digit.distal]

    for bone_name, bone in zip(bone_names, bones):
        pj = bone.prev_joint
        nj = bone.next_joint

        output.extend([
            str(pj.x), str(pj.y), str(pj.z),
            str(nj.x), str(nj.y), str(nj.z)
        ])

    # Extended state
    is_extended = digit.is_extended
    output.append("1" if is_extended else "0")

# No second hand
while len(output) < 301:
    output.append("0")

row = ",".join(output)

```

```

# Convert row to single-row DataFrame
df = pd.DataFrame([row.split(",")], dtype="float32")

# Normalisation.py again
# Zero second hand if only one hand
df.loc[df[self.num_hands] == 1, self.hand2_start:self.hand2_end] = 0

# Palm-relative Normalisation
# Hand 1
for i in range(0, len(self.hand1_cols), 3):
    x_col = self.hand1_cols[i]
    y_col = self.hand1_cols[i + 1]
    z_col = self.hand1_cols[i + 2]

    df[x_col] = df[x_col] - df[self.palm1_x]
    df[y_col] = df[y_col] - df[self.palm1_y]
    df[z_col] = df[z_col] - df[self.palm1_z]

df[self.palm1_x] = 0
df[self.palm1_y] = 0
df[self.palm1_z] = 0

# Hand 2
for i in range(0, len(self.hand2_cols), 3):
    x_col = self.hand2_cols[i]
    y_col = self.hand2_cols[i + 1]
    z_col = self.hand2_cols[i + 2]

    if z_col >= df.shape[1]:
        break

    df[x_col] = df[x_col] - df[self.palm2_x]
    df[y_col] = df[y_col] - df[self.palm2_y]
    df[z_col] = df[z_col] - df[self.palm2_z]

if self.palm2_z < df.shape[1]:
    df[self.palm2_x] = 0
    df[self.palm2_y] = 0
    df[self.palm2_z] = 0

features = df.iloc[:, 4:].copy()

# Z-score Normalisation

```

```

zscore_feature_cols = [col - 4 for col in self.zscore_cols]

feature_array = features.to_numpy(dtype="float32")

feature_array[0, zscore_feature_cols] = (
    feature_array[0, zscore_feature_cols] - self.means
) / self.stds

# Reshape
feature_array = feature_array.reshape(1, feature_array.shape[1], 1)

# Prediction
predictions = self.model.predict(feature_array, verbose=0)

predicted_class = int(np.argmax(predictions, axis=1)[0])
confidence = float(np.max(predictions))

predicted_label = self.label_map[predicted_class]

print("-----")
print("Predicted Sign:", predicted_label)
print("Confidence:", round(confidence, 4))
print("-----")

def main():
    my_listener = MyListener()

    connection = leap.Connection()
    connection.add_listener(my_listener)

    running = True

    with connection.open():
        connection.set_tracking_mode(leap.TrackingMode.Desktop)

        while running:
            time.sleep(1)

if __name__ == "__main__":
    main()

```